Cristian S. Garcés Programmer Task

General Idea

The project was designed with the primary goal of replicating the Grand Theft Auto gaming experience, paying special attention to fluid movement in a 3D environment and the accessibility of a quick “selection wheel” inventory. The code architecture was organized by modules (features), ensuring a clear separation of responsibilities and facilitating maintenance.

1. Module: Character Movement

This module focuses on providing smooth and responsive player interaction with the 3D environment, using standard Unity tools:

Input System: Unity's InputSystem was implemented as the base layer for capturing user input (keyboard, gamepad). A dedicated component (“PlayerInputReader”) converts these raw input events, isolating the control logic.

Player Logic and Animation: The player movement component consumes these events to apply movement in the world. Animation is handled with a Blend Tree, which takes the magnitude of the player's input to perform a smooth transition between states (rest, walk, run), achieving natural visual behavior.

Dynamic Camera: The experience is complemented by Cinemachine, used to manage the third-person camera, ensuring that it follows the character smoothly and automatically.

2. Module: Inventory System

The inventory was designed with a scalable structure, prioritizing space limitations to fit the GTA style:

Class Architecture: A Generic Parent Class was created to contain all the basic logic (adding, removing, managing items). Then, a Child Class (LimitedInventory) inherits these functionalities and imposes the restriction of 8 spaces, the limit necessary for the future implementation of the GTA-style “selection wheel.”

Connection to the Interface (UI): The inventory logic connects directly to its respective UI classes for visual feedback. This connection allows the user not only to view the items, but also to manipulate them directly from the interface, implementing essential functionalities such as:

- Drag and Drop: To reorder or swap items between slots.
- Deletion: To discard items from the inventory through the interface.

3. Module: Data Persistence

To ensure that player progress is maintained, a simple save/load system was implemented:

Serialization to JSON: The system uses JSON files to save information to the hard drive. Only essential information is saved: the unique ID of the item (the key to recreating it) and its exact position within the inventory.

Inventory Reconstruction: The key to persistence is the script (LoadDroppedItems) that checks the objects in the world and verifies if their ID is found in the JSON file. If it is found, it simply removes the instance from the world and adds it to the inventory.

The game saves the data in the JSON file every time an item is added, dropped, or swapped, as well as when the application is closed.

4. Module: Equip items

As a final step, the logic for equip inventory items has been added.

Hover Feedback: The “hover” function (moving the cursor over) was implemented on the interface items. When this action is detected, the system extracts the data for the item in question.

Prefab Instantiation: This hover action is used as a trigger to instantiate the corresponding visual prefab of the item in the hand or in the proximity of the character, offering an immediate preview or equipping it directly.